



RapFlow-TTS: Rapid and High-Fidelity Text-to-Speech with Improved Consistency Flow Matching

Hyun Joon Park^{1,2,*}, Jeongmin Liu¹, Jin Sob Kim², Jeong Yeol Yang², Sung Won Han², Eunwoo Song^{1,3}

¹NAVER Cloud, Republic of Korea

²School of Industrial and Management Engineering, Korea University, Republic of Korea

³Artificial Intelligence Institute, Seoul National University, Republic of Korea

winddori2002@korea.ac.kr

Abstract

We introduce RapFlow-TTS, a rapid and high-fidelity TTS acoustic model that leverages velocity consistency constraints in flow matching (FM) training. Although ordinary differential equation (ODE)-based TTS generation achieves natural-quality speech, it typically requires a large number of generation steps, resulting in a trade-off between quality and inference speed. To address this challenge, RapFlow-TTS enforces consistency in the velocity field along the FM-straightened ODE trajectory, enabling consistent synthetic quality with fewer generation steps. Additionally, we introduce techniques such as time interval scheduling and adversarial learning to further enhance the quality of the few-step synthesis. Experimental results show that RapFlow-TTS achieves high-fidelity speech synthesis with a 5- and 10-fold reduction in synthesis steps than the conventional FM- and score-based approaches, respectively.

Index Terms: rapid, text-to-speech, consistency model, flow matching, adversarial learning

1. Introduction

Text-to-speech (TTS), also known as speech synthesis, aims to synthesize high-fidelity speech, given an input text [1, 2, 3]. Among the various generative modeling approaches [4, 5, 6, 7, 8], ordinary differential equations (ODE)-based models [9, 10] have become strong solutions for outstanding TTS. One representative method involves diffusion models using stochastic differential equations (SDE) [11, 12], e.g., Grad-TTS [9]. It trains a score network with an SDE-based diffusion process and solves the probability flow ODE for speech synthesis. However, ODE trajectories obtained by diffusion models are complicated [13, 14], requiring numerous steps to generate high-quality speech [15]. Since TTS functions as a user interaction channel, the slow inference speed caused by numerous diffusion steps is a major drawback of diffusion-based TTS.

To resolve this limitation, researchers investigated improved ODE-based models [13, 14, 16, 17] for TTS. For example, Comospeech [18] applied consistency distillation [17] to transfer the knowledge of a diffusion teacher model to a consistent student model. As the consistency constraint maps any point on the ODE trajectory to the endpoint, i.e., the target data, the student model can consistently produce high-quality speech regardless of the number of diffusion steps. However, the complexity of diffusion-based ODE trajectories limited the effective construction of consistency models along those trajectories.

Meanwhile, flow matching (FM), which learns the vector field for the ODE and probability path, is another approach for improving ODE-based models. Using an optimal transport

plan that makes the probability path linear in time [13, 14], the ODE trajectory tends to be straight, making the few-step performance of FM superior to that of the diffusion models. In TTS frameworks, Matcha-TTS [19] and VoiceFlow [15] adopted the FM-based straight ODE trajectory, demonstrating high-quality speech synthesis with even fewer generation steps than conventional diffusion-based models. However, completely generalizing the real distribution to a straight ODE trajectory is difficult; thus, the quality of the few-step synthesized speech has not yet reached that of the ground truth. In summary, previous ODE-based TTS approaches failed to simultaneously satisfy both aspects, i.e., inference speed and speech synthesis quality.

In this study, we present RapFlow-TTS, a rapid and high-fidelity TTS acoustic model that effectively addresses the limitations above. Inspired by advancements in image generation, we adopt consistency FM [20], a novel FM method that explicitly enforces self-consistency in the velocity field. By imposing consistency constraints on the velocity values, consistency FM directly defines straight flows from different time points to a common endpoint. This allows RapFlow-TTS to learn how to produce consistent outputs along the straight trajectory more effectively, unlike diffusion-based methods with complicated trajectories, yielding a more effective consistency model. Consequently, high-quality speech synthesis is achievable with only a few generation steps. To the best of our knowledge, RapFlow-TTS is the first TTS framework based on consistency FM.

Furthermore, we propose several improved training techniques to enhance the performance, including shared dropout, Huber loss, time-interval scheduling, and adversarial learning, none of which have been explored in the context of consistency FM. Experimental results show that the proposed RapFlow-TTS can synthesize more natural speech in just 2 steps, reducing generation steps by 5 to 10 times compared to previous ODE-based methods. Our code and demos are available here.¹

2. Consistency Flow Matching

Here, we review the consistency FM [20] method for training the RapFlow-TTS. Given a time index $t \in [0, 1]$, sampled from a uniform distribution, FM learns a ground truth vector field u_t to build the probability path $p_t(x_t)$ from the random noise $x_0 \sim p_0$ to target data $x_1 \sim p_1$. The vector field is defined as an ODE for a flow ϕ_t , where ϕ_t is a function that transforms data over time, as follows:

$$\frac{d}{dt} \phi_t(x) = u_t(\phi_t(x)); \quad \phi_0(x) = x, \quad (1)$$

The solution of Eq. (1), i.e., flow, represents the sampling path. For training FM models, a simple regression with the ground-

* Work performed as an intern in the Voice team, NAVER Cloud.

¹<https://tts-demo.github.io/rap.github.io/>

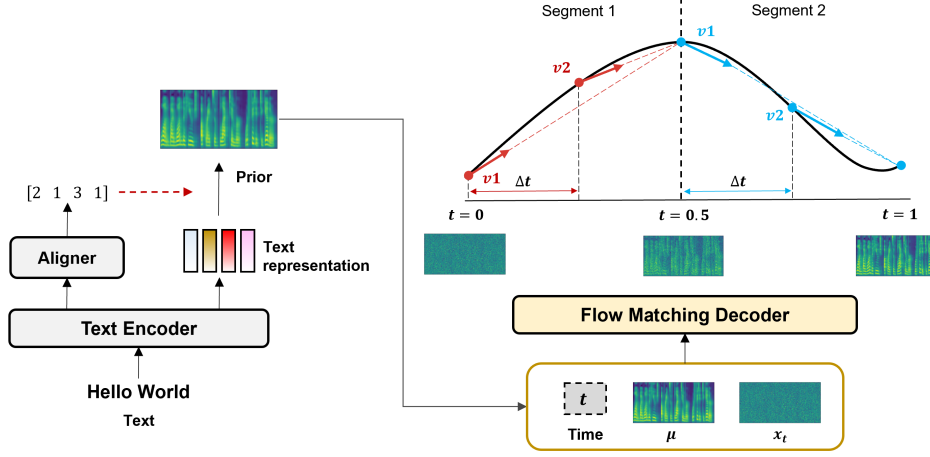


Figure 1: Overview of RapFlow-TTS with multi-segment consistency flow matching.

truth vector field is applied as follows:

$$\mathcal{L}_{FM} = \|v_\theta(t, x_t) - u(t, x_t)\|_2^2, \quad (2)$$

where $v_\theta(t, x_t)$ denotes the learnable vector field network.

In contrast to the FM, consistency FM trains the vector field by ensuring that any point on the trajectory reaches the same endpoint with the same velocity. Let the linearly interpolated noise sample at time t be $x_t = tx_1 + (1-t)x_0$. Its objective function is composed of a straight flow loss $\mathcal{L}_{sf}$ and a velocity consistency loss $\mathcal{L}_{vc}$ as follows:

$$\begin{aligned} \mathcal{L}_{cfm} &= \mathcal{L}_{sf} + \alpha \mathcal{L}_{vc}, \\ \mathcal{L}_{sf} &= \|f_\theta(t, x_t) - f_\theta(t + \Delta t, x_{t+\Delta t})\|_2^2, \\ \mathcal{L}_{vc} &= \|v_\theta(t, x_t) - v_\theta(t + \Delta t, x_{t+\Delta t})\|_2^2, \\ f_\theta(t, x_t) &= x_t + (1-t) \times v_\theta(t, x_t), \end{aligned} \quad (3)$$

where θ^- denotes model parameters with a stop gradient; α denotes a loss weight; and Δt denotes a time interval. Since f_θ guides x_t to the estimated endpoint of the trajectory, the straight flow loss constrains consistency from the trajectory viewpoint, ensuring straight flow. In contrast, the velocity consistency loss directly enforces the consistency of the vector field. This allows the consistency FM to learn how to produce consistent outputs along the straightened trajectory, accurately estimating the target distribution even with a few-step generation.

3. RapFlow-TTS

3.1. TTS with Consistency Flow Matching

For RapFlow-TTS, we follow the network design of Matcha-TTS [19] thanks to its fast and lightweight properties. As shown in Figure 1, RapFlow-TTS consists of a text encoder, an aligner, and a flow matching decoder. The text encoder extracts the context representations from the input text, and the aligner maps these representations to prior mel-spectrograms μ . The text encoder and aligner are trained using duration loss $\mathcal{L}_{dur}$ and prior loss $\mathcal{L}_{prior}$ based on the MAS algorithm as in [3, 9].

Consistency FM trains the FM decoder to construct a probability path p_t that transports the random noise $x_0 \sim p_0$ to target mel-spectrogram’s distribution $x_1 \sim p_1$. We condition the consistency FM objective with the prior μ to synthesize speech

corresponding to the given input text. We also adopt a multi-segment objective for flexible transportation between complex distributions [20]. In detail, the time range $t \in [0, 1]$ is divided into S segments of equal size, where each segment corresponds to the time range $[i/S, (i+1)/S]$ for $i = 0, 1, \dots, S-1$. Then, the objective function of segmented consistency FM is expressed as follows:

$$\begin{aligned} \mathcal{L}_{cfm} &= \mathcal{L}_{sf} + \alpha \mathcal{L}_{vc}, \\ \mathcal{L}_{sf} &= \|f_\theta^i(t, x_t, \mu) - f_\theta^i(t + \Delta t, x_{t+\Delta t}, \mu)\|_2^2, \\ \mathcal{L}_{vc} &= \|v_\theta^i(t, x_t, \mu) - v_\theta^i(t + \Delta t, x_{t+\Delta t}, \mu)\|_2^2, \\ f_\theta^i(t, x_t, \mu) &= x_t + ((i+1)/S - t) \times v_\theta^i(t, x_t, \mu), \end{aligned} \quad (4)$$

Consequently, RapFlow-TTS learns straight flow with a consistent vector field in each segment, effectively representing the data distribution by a piecewise linear trajectory.

In practice, we apply a two-stage training strategy to effectively optimize the RapFlow-TTS model. In the first stage (the first N epochs), we aim to train RapFlow-TTS to have a straight flow by focusing on the trajectory viewpoint. To achieve it, we optimize RapFlow-TTS using only the straight flow loss $\mathcal{L}_{sf}$ in Eq. (4), slightly modifying the loss term as $\|f_\theta^i(t, x_t, \mu) - x^i\|_2^2$, where x^i is the ground-truth endpoint at each segment, defined as $x^i = (i+1)/S \times x_1 + (1 - (i+1)/S) \times x_0$. Because this loss guides the ground-truth endpoint on the trajectory, RapFlow-TTS can learn the vector field with a straight flow that represents the real data distribution.

In the second stage (the following N epochs), we train RapFlow-TTS to have a straight flow with a consistent vector field using the entire $\mathcal{L}_{cfm}$ in Eq. (4). By training consistency FM on a straight ODE trajectory, RapFlow-TTS can learn consistency more easily than diffusion-based methods, constructing a more effective consistency model. As a result, RapFlow-TTS has the properties of consistency and straight flow, thus synthesizing high-quality speech in fewer synthesis steps.

3.2. Improved techniques for RapFlow-TTS

We introduce several techniques to improve the consistency FM model for RapFlow-TTS. All of these methods enhance few-step synthesis performance, and their effectiveness will be further analyzed in Section 4.3.

Encoder Freeze We freeze the encoder and optimize only $\mathcal{L}_{cfm}$ for training efficiency in the second stage. It also stabilizes consistency training by keeping the condition μ frozen.

Shared Dropout Although diffusion-based consistency models and rectified flow [21, 22] proved that applying dropout improves the sample quality, the effect of dropout in consistency FM has not been investigated. Based on [21], while training the consistency FM model in the second stage, we apply dropout using the same random state for v_θ and $v_{\theta-}$. Empirically, we use a dropout ratio of 0.05.

Pseudo-Huber Loss Our loss in Eq. (4) is composed of the ℓ_2 metric. However, the ℓ_2 metric imposes a large penalty on outliers, which can lead to an imbalanced loss with respect to time t , potentially increasing the gradient variance [23]. To mitigate this issue, we adopt the pseudo-Huber metric, which is less sensitive to outliers. We follow the implemented metric from [21], defined as $d(x, y) = \sqrt{\|x - y\|_2^2 + c^2} - c$, where $c = 0.00054\sqrt{d}$ and d is the data dimension size.

Delta Scheduling The term Δt in Eq. (4) is a parameter representing the time interval between two points on the trajectory, similar to the discretization size parameter in diffusion-based consistency models [17]. This implies that a small Δt reduces the bias but increases the variance of the model, and vice versa for a large Δt . Since a small bias and large variance are desirable near the end of the training, inspired by [17], we employ scheduling that progressively reduces Δt during the training. To train the model uniformly across diverse Δt , we utilize linear step scheduling. In practice, Δt decreases from 0.1 to 0.001 over K intervals, and each Δt is used during N/K epochs.

Adversarial Learning To enhance synthesis quality, we utilize adversarial learning after the second stage. We adopt MSE-based adversarial loss $\mathcal{L}_{adv}$ [24] and feature matching loss $\mathcal{L}_{fm}$ [25] on the mel-spectrogram levels using a Conv2d discriminator [26]. Furthermore, we extend it to multi-segment adversarial learning for consistency FM. Let x^i and $\hat{x}^i$ be the ground-truth and estimated endpoint at each segment, where each is obtained by $(i + 1)/S \times x_1 + (1 - (i + 1)/S) \times x_0$ and $f_\theta^i(t, x_t, \mu)$, respectively. Then, the objective function is defined as follows:

$$\begin{aligned} \mathcal{L}_{adv} &= \|D(\hat{x}^i)\|_2^2 + \|1 - D(x^i)\|_2^2, \\ \mathcal{L}_{fm} &= \sum_{l=1}^L \|D^l(\hat{x}^i) - D^l(x^i)\|_1, \end{aligned} \quad (5)$$

where D indicates the discriminator; D^l denotes the l^{th} feature map of D . By applying multi-segment adversarial learning, we improve the endpoint quality of each segment, yielding better few-step performance. Finally, in this stage, we optimize $\mathcal{L}_{cfm}$, $\mathcal{L}_{adv}$, and $\mathcal{L}_{fm}$ with the ratio of 3, 1, and 2.

4. Experiments

4.1. Experiment Setup

Dataset To verify RapFlow-TTS, we conducted experiments on the LJSpeech [27] and VCTK [28] datasets. The LJSpeech is a single-speaker English dataset containing about 24 hours of audio clips. For the dataset split, we followed the settings of previous work in [2]. The VCTK is a multi-speaker English dataset composed of about 44 hours of speech recorded by 110 speakers. We randomly split the train, valid, and test sets in the ratios of 80%, 10%, and 10%, respectively. To make the models compatible with the HiFiGAN vocoder used in the experiments, we followed the acoustic settings of HiFiGAN-V1 [29].

Baselines For comparison, we set the following systems as baselines: 1) *GT*, ground truth; 2) *VOC*, ground truth recon-

structed by the vocoder; 3) *FastSpeech2* [7], Transformer-based TTS; 4) *Grad-TTS* [9], diffusion-based TTS; 5) *Comospeech* [18], consistency model with diffusion-based TTS; 6) *VoiceFlow* [15], rectified flow for TTS; and 7) *Matcha-TTS* [19], flow matching for TTS. We recorded the performance of the baselines after training with their official codes. Since there is no official code for FastSpeech2, we used Meta’s checkpoint.²

Implementation Details For training, we set the number of epochs N to 1400 and 1000 for the LJSpeech and VCTK datasets, respectively. We additionally used 150 and 50 epochs for adversarial learning, respectively. We used Adam optimizer with a learning rate of 1×10^{-4} and batch size of 16. Regarding the hyperparameters of consistency FM, we set the multi-segment size S to 2, the bin size K for delta scheduling to 8, and the loss weight α in Eq. (4) to 1×10^{-5} . The encoder freeze and shared dropout were applied as defaults, and the results with other techniques were shown through $\dagger$ notation in the experiments. In addition, we followed the network hyperparameters of Matcha-TTS. Finally, we applied an Euler solver to the ODE-based TTS models to synthesize speech. To investigate the few-step performance of ODE-based models, we set the number of function evaluations (NFE) to 2. To ensure reasonable performance for models that do not build a consistency model, such as Matcha-TTS and Grad-TTS, we also recorded their performances using NFE of 10 and 25, respectively.

Evaluation Metrics We assessed the results in terms of speech intelligibility, naturalness, and real-time factor (RTF). For intelligibility, we measured word error rate (WER) by applying the Whisper-M-based ASR model [30] to the synthesized speech. Regarding naturalness, we conducted a mean opinion score (MOS) test. Twenty participants were asked to assess the naturalness of the synthesized speech by scoring it from 1 to 5 ranges. Twenty utterances were randomly selected from each system. In addition, we used NISQA [31], a model-based naturalness assessment method, for ablation studies. We provided evaluation results with a 95% confidence interval. Finally, we recorded the RTF, the ratio between the model synthesis time and duration of the synthesized speech, using a single Tesla V100 GPU to compare the inference speeds.

4.2. Experimental Results

Table 1 shows the evaluation results of which findings can be analyzed as follows: Firstly, the conventional score-based (Grad-TTS) and FM-based (VoiceFlow and Matcha-TTS) models exhibited significant degradation in naturalness (MOS) with the 2-step synthesis compared to the many-step synthesis. By training with the consistency FM objective, the proposed RapFlow-TTS achieved superior performance with a 5- to 10-fold reduction in NFE than previous methods. In addition, it outperformed Comospeech in both speech intelligibility (WER) and naturalness. These results indicate that training consistency on a straightened ODE path leads to an effective consistency model, thereby improving few-step performance. However, the limited performance of the 2-step synthesis of the previous methods is attributed to the construction of consistency models on complex ODE paths (Comospeech) or the absence of consistency in FM-based methods (Matcha-TTS and VoiceFlow). Secondly, the improved techniques in RapFlow-TTS[†] contributed to significant enhancements in both the intelligibility and naturalness aspects. This is an encouraging result considering that the 2-step speed of RapFlow-TTS is comparable to the speed

²<https://github.com/facebookresearch/fairseq>

[†] indicates the improved techniques are applied.

Table 1: Evaluation results on the LJSpeech test set.

Model	Params	NFE	RTF	WER	MOS
GT	-	-	-	3.31 $\pm$ 0.88	4.42 $\pm$ 0.08
VOC	-	-	0.022	3.43 $\pm$ 0.95	4.30 $\pm$ 0.08
Grad-TTS	14.8M	25	0.133	4.32 $\pm$ 0.82	3.78 $\pm$ 0.09
VoiceFlow	14.8M	10	0.069	4.04 $\pm$ 0.81	3.42 $\pm$ 0.09
Matcha-TTS	18.2M	10	0.056	3.28 $\pm$ 0.78	3.83 $\pm$ 0.09
FastSpeech2	41.2M	1	0.029	5.58 $\pm$ 1.04	3.35 $\pm$ 0.10
Comospeech	14.8M	2	0.034	5.41 $\pm$ 0.93	3.19 $\pm$ 0.10
Grad-TTS	14.8M	2	0.034	4.06 $\pm$ 0.79	1.92 $\pm$ 0.07
VoiceFlow	14.8M	2	0.034	4.00 $\pm$ 0.79	3.05 $\pm$ 0.09
Matcha-TTS	18.2M	2	0.031	3.15 $\pm$ 0.76	3.32 $\pm$ 0.10
RapFlow-TTS	18.2M	2	0.031	3.33 $\pm$ 0.80	3.89 $\pm$ 0.09
RapFlow-TTS [†]	18.2M	2	0.031	3.11 $\pm$ 0.76	4.01 $\pm$ 0.09

Table 2: Ablation study results on the LJSpeech test set: All the samples in (A)-(H) models were synthesized with 2 NFE steps.

Model	WER	NISQA
(A): Straight flow (Stage 1)	3.19 $\pm$ 0.79	3.46 $\pm$ 0.04
(B): (A) + Consistency FM	3.51 $\pm$ 0.80	3.71 $\pm$ 0.05
(C): (B) + Encoder freeze	3.48 $\pm$ 0.80	3.73 $\pm$ 0.05
(D): (C) + Shared dropout (RapFlow-TTS)	3.33 $\pm$ 0.80	3.78 $\pm$ 0.05
(E): (D) + Huber loss	3.42 $\pm$ 0.78	3.87 $\pm$ 0.04
(F): (D) + Delta scheduling	3.28 $\pm$ 0.78	3.90 $\pm$ 0.05
linear $\rightarrow$ exp schedule	3.34 $\pm$ 0.79	3.78 $\pm$ 0.05
(G): (D) + Adversarial learning	3.24 $\pm$ 0.78	4.19 $\pm$ 0.04
(H): Putting it together (RapFlow-TTS [†])	3.11 $\pm$ 0.76	4.25 $\pm$ 0.04
w/ NFE = 10	3.41 $\pm$ 0.74	4.28 $\pm$ 0.04
w/ NFE = 25	3.47 $\pm$ 0.80	4.29 $\pm$ 0.04

of FastSpeech2, which is generally adopted as a benchmark for fast TTS models. In summary, RapFlow-TTS synthesizes high-quality speech with fewer steps, achieved through consistency FM and improved training strategies, mitigating the inference speed limitations of previous ODE-based TTS.

4.3. Further Experiments

Ablation Studies In the first block of Table 2, we analyzed the results of consistency FM and default techniques such as encoder freeze and shared dropout. Experiment (A) represents the results from stage 1, described in Section 3.1, where only a straight flow was trained without consistency. Experiment (B) presents the results of stage 2, in which the consistency FM was additionally trained at the stage-1 model. Despite some decrease in speech intelligibility, we observed a notable improvement in naturalness. It demonstrates that building a consistency model on a straight ODE trajectory is effective in enhancing the quality of the few-step synthesis. Meanwhile, experiment (C) and (D) show the results obtained using encoder freeze and shared dropout, respectively. Although the impact of the encoder freeze on the performance was not substantial, we empirically observed that it increased training speed by 1.3 times. Shared dropout improved the robustness of the consistency FM model, leading to enhanced performance across all metrics.

In the second block of Table 2, we investigated the effectiveness of other improved techniques for RapFlow-TTS as follows: Firstly, in experiment (E), replacing the ℓ_2 metric with the Huber metric improved the speech naturalness by reducing the gradient variance. Although we observed a slight decrease in WER performance, we adopted this strategy since we pri-

Table 3: Evaluation results on the VCTK test set.

Model	NFE	WER	MOS
GT	-	1.79 $\pm$ 0.21	4.42 $\pm$ 0.07
VOC	-	2.10 $\pm$ 0.23	4.06 $\pm$ 0.09
Straight flow (Stage 1)	2	2.01 $\pm$ 0.24	3.53 $\pm$ 0.09
Straight flow (Stage 1)	10	2.52 $\pm$ 0.28	3.83 $\pm$ 0.09
RapFlow-TTS	2	3.25 $\pm$ 0.31	3.83 $\pm$ 0.10
RapFlow-TTS [†]	2	2.01 $\pm$ 0.23	4.28 $\pm$ 0.08

oritized naturalness enhancement. Secondly, delta scheduling (experiment (F)) improved both WER and NISQA. It indicates that the strategy, progressively reducing Δt during training, is effective for performance improvement by minimizing the bias at the end of training. In addition, the performance degradation observed when using exponential scheduling suggests that linear scheduling, which uniformly trains various Δt during training, is appropriate. Lastly, in experiment (G), we validated the effects of adversarial learning. This strategy yielded the most significant improvements compared to previous techniques. By learning the ground-truth characteristics at the end-points of each segment, RapFlow-TTS with adversarial learning can synthesize high-quality speech even in 2-step.

Finally, in experiment (H), where we combined all improved techniques, RapFlow-TTS achieved the best WER and NISQA. It suggests that each technique contributes to performance improvement as its unique method without redundancy. Furthermore, we utilized NFE of 10 and 25 for experiment (H) to analyze the performance of RapFlow-TTS depending on NFE. We observed that naturalness slightly improved as NFE increased, while speech intelligibility deteriorated. This can be attributed to the characteristics of consistency models [18], where performance improvements are minimal or even degrade in many-step synthesis. In future work, we further analyze this issue and explore potential improvements.

Results on the multi-speaker dataset To verify RapFlow-TTS on the multi-speaker dataset, we conducted experiments on the VCTK dataset. Considering the increase in dataset size and number of speakers, we raised the shared dropout to 0.1 for model robustness. As depicted in Table 3, RapFlow-TTS without improved techniques demonstrated lower speech intelligibility compared to the stage-1 model. However, in terms of naturalness, it significantly outperformed the stage-1 model in 2-step synthesis, even achieving a quality comparable to the stage-1 model’s 10-step performance. In addition, the improved techniques substantially enhanced both speech intelligibility and naturalness, enabling the synthesis of high-quality and intelligible speech in only 2 steps. These experimental results indicate that RapFlow-TTS can achieve an outstanding few-step synthesis performance even on multi-speaker datasets.

5. Conclusion

In this study, we proposed RapFlow-TTS, the ODE-based TTS model capable of fast and high-fidelity speech synthesis. Based on consistency FM, RapFlow-TTS constructs a consistency model on a straight flow. By leveraging both the straight trajectory and the velocity-consistency property, high-quality speech synthesis was achievable in significantly fewer steps. In addition, training techniques such as time interval scheduling and adversarial learning further enhanced performance. As a result, RapFlow-TTS outperformed existing ODE-based models and effectively overcame the trade-off between quality and speed.

6. Acknowledgements

This research was supported by Voice team, NAVER Cloud, Republic of Korea. This research was supported by the BK21 FOUR funded by the Ministry of Education of Korea and National Research Foundation of Korea. This research was also results of a study on the “Leaders in INdustry-university Co-operation 3.0” Project, supported by the Ministry of Education and National Research Foundation of Korea.

7. References

- [1] Y. Wang, R. Skerry-Ryan, D. Stanton, Y. Wu, R. J. Weiss, N. Jaitly, Z. Yang, Y. Xiao, Z. Chen, S. Bengio, Q. V. Le, Y. Agiomyriannakis, R. Clark, and R. A. Saurous, “Tacotron: Towards end-to-end speech synthesis,” in *Proc. Interspeech*, 2017, pp. 4006–4010.
- [2] J. Shen, R. Pang, R. J. Weiss, M. Schuster, N. Jaitly, Z. Yang, Z. Chen, Y. Zhang, Y. Wang, R. Skerry-Ryan *et al.*, “Natural TTS synthesis by conditioning Wavenet on mel spectrogram predictions,” in *Proc. ICASSP*, 2018, pp. 4779–4783.
- [3] J. Kim, S. Kim, J. Kong, and S. Yoon, “Glow-TTS: A generative flow for text-to-speech via monotonic alignment search,” in *Proc. NeurIPS*, vol. 33, 2020, pp. 8067–8077.
- [4] Y. Ren, Y. Ruan, X. Tan, T. Qin, S. Zhao, Z. Zhao, and T.-Y. Liu, “FastSpeech: Fast, robust and controllable text to speech,” in *Proc. NeurIPS*, vol. 32, 2019.
- [5] C. Miao, S. Liang, M. Chen, J. Ma, S. Wang, and J. Xiao, “Flow-TTS: A non-autoregressive network for text to speech based on flow,” in *Proc. ICASSP*, 2020, pp. 7209–7213.
- [6] N. Li, S. Liu, Y. Liu, S. Zhao, M. Liu, and M. Zhou, “Neural speech synthesis with Transformer network,” in *Proc. AAAI*, 2019, pp. 6706–6713.
- [7] Y. Ren, C. Hu, X. Tan, T. Qin, S. Zhao, Z. Zhao, and T.-Y. Liu, “FastSpeech 2: Fast and high-quality end-to-end text to speech,” in *Proc. ICLR*, 2021.
- [8] M. Jeong, J. Kim, J. Kong, and J. Son, “Diff-TTS: A denoising diffusion model for text-to-speech,” in *Proc. Interspeech*, 2021, pp. 3605–3609.
- [9] V. Popov, I. Vovk, V. Gogoryan, T. Sadekova, and M. Kudinov, “Grad-TTS: A diffusion probabilistic model for text-to-speech,” in *Proc. ICML*, 2021, pp. 8599–8608.
- [10] H. J. Park, J. S. Kim, W. Shin, and S. W. Han, “DEX-TTS: Diffusion-based EXpressive Text-to-Speech with style modeling on time variability,” *arXiv preprint arXiv:2406.19135*, 2024.
- [11] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, “Score-based generative modeling through stochastic differential equations,” in *Proc. ICLR*, 2021.
- [12] T. Karras, M. Aittala, T. Aila, and S. Laine, “Elucidating the design space of diffusion-based generative models,” in *Proc. NeurIPS*, vol. 35, 2022, pp. 26 565–26 577.
- [13] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, “Flow matching for generative modeling,” in *Proc. ICLR*, 2023.
- [14] A. Tong, K. FATRAS, N. Malkin, G. Hugué, Y. Zhang, J. Rector-Brooks, G. Wolf, and Y. Bengio, “Improving and generalizing flow-based generative models with minibatch optimal transport,” *Transactions on Machine Learning Research*, 2024.
- [15] Y. Guo, C. Du, Z. Ma, X. Chen, and K. Yu, “Voiceflow: Efficient text-to-speech with rectified flow matching,” in *Proc. ICASSP*, 2024, pp. 11 121–11 125.
- [16] X. Liu, C. Gong, and qiang liu, “Flow straight and fast: Learning to generate and transfer data with rectified flow,” in *Proc. ICLR*, 2023.
- [17] Y. Song, P. Dhariwal, M. Chen, and I. Sutskever, “Consistency models,” in *Proc. ICML*, 2023, pp. 32 211–32 252.
- [18] Z. Ye, W. Xue, X. Tan, J. Chen, Q. Liu, and Y. Guo, “Como-speech: One-step speech and singing voice synthesis via consistency model,” in *Proc. ACM Multimedia*, 2023, pp. 1831–1839.
- [19] S. Mehta, R. Tu, J. Beskow, É. Székely, and G. E. Henter, “Matcha-TTS: A fast tts architecture with conditional flow matching,” in *Proc. ICASSP*, 2024, pp. 11 341–11 345.
- [20] L. Yang, Z. Zhang, Z. Zhang, X. Liu, M. Xu, W. Zhang, C. Meng, S. Ermon, and B. Cui, “Consistency flow matching: Defining straight flows with velocity consistency,” *arXiv preprint arXiv:2407.02398*, 2024.
- [21] Y. Song and P. Dhariwal, “Improved techniques for training consistency models,” in *Proc. ICLR*, 2024.
- [22] B. Kim, Y.-G. Hsieh, M. Klein, marco cuturi, J. C. Ye, B. Kavar, and J. Thornton, “Simple reflow: Improved techniques for fast flow models,” in *Proc. ICLR*, 2025.
- [23] S. Lee, Z. Lin, and G. Fanti, “Improving the training of rectified flows,” in *Proc. NeurIPS*, 2024.
- [24] X. Mao, Q. Li, H. Xie, R. Y. Lau, Z. Wang, and S. Paul Smolley, “Least squares generative adversarial networks,” in *Proc. ICCV*, 2017, pp. 2794–2802.
- [25] T. Salimans, I. Goodfellow, W. Zaremba, V. Cheung, A. Radford, and X. Chen, “Improved techniques for training gans,” in *Proc. NeurIPS*, vol. 29, 2016.
- [26] Y. A. Li, C. Han, and N. Mesgarani, “StyleTTS: A style-based generative model for natural and diverse text-to-speech synthesis,” *arXiv preprint arXiv:2205.15439*, 2022.
- [27] K. Ito and L. Johnson, “The LJ speech dataset,” <https://keithito.com/LJ-Speech-Dataset/>, 2017.
- [28] J. Yamagishi, C. Veaux, and K. MacDonald, “CSTR VCTK corpus: English multi-speaker corpus for CSTR voice cloning toolkit (version 0.92),” 2019.
- [29] J. Kong, J. Kim, and J. Bae, “HiFi-GAN: Generative adversarial networks for efficient and high fidelity speech synthesis,” in *Proc. NeurIPS*, vol. 33, 2020, pp. 17 022–17 033.
- [30] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust speech recognition via large-scale weak supervision,” in *Proc. ICML*, 2023, pp. 28 492–28 518.
- [31] G. Mittag, B. Naderi, A. Chehadi, and S. Möller, “NISQA: A deep CNN-self-attention model for multidimensional speech quality prediction with crowdsourced datasets,” in *Proc. Interspeech*, 2021, pp. 2127–2131.